

WHITE BOX TESTING

TESTING DAN IMPLEMENTASI SISTEM

A solid orange horizontal bar at the bottom of the slide.

Tiga Tingkatan Testing secara Umum

Unit testing

Testing penulisan kode-kode program dalam satuan unit terkecil secara individual.

System Testing

Proses testing pada sistem terintegrasi untuk melakukan verifikasi bahwa sistem telah sesuai spesifikasi.

Acceptance Testing

Testing formal yang dilakukan untuk menentukan apakah sistem telah memenuhi kriteria penerimaan dan memberdayakan pelanggan untuk menentukan apakah sistem dapat diterima atau tidak.

Praktik *unit testing* secara umum

Tujuan: Konfirmasi bahwa modul telah dikode dengan benar.

Pelaku: Biasanya programmer.

Apa yang dites:

- Fungsi (*Black Box*).
- Kode (*White Box*).
- Kondisi ekstrim dan batasan-batasan.

Kapan selesai: Biasanya saat programmer telah merasa puas dan tidak diketahui lagi kesalahan.

Alat bantu: Tidak biasa digunakan.

Data: Biasanya tidak didata.

Praktik *system testing* secara umum

Tujuan: Merakit modul menjadi suatu sistem yang bekerja dan menentukan kesiapan untuk melakukan *Acceptance Test*.

Pelaku: Pemimpin tim atau grup tes.

Apa yang dites:

- Kebutuhan dan fungsi sistem.
- Antarmuka sistem.

Kapan selesai: Biasanya bila mayoritas kebutuhan telah sesuai dan tidak ada kesalahan mayor yang ditemukan.

Alat bantu: Sistem pustaka dan pustaka *test case*; Generator, komparator dan simulator data testing.

Data: Data kesalahan yang ditemukan; Test case.

Praktik *acceptance testing* secara umum

Tujuan: Mengevaluasi kesiapan untuk digunakan.

Pelaku: Pengguna akhir atau agen.

Apa yang dites:

- Fungsi mayor.
- Dokumentasi.
- Prosedur.

Kapan selesai: Biasanya bila pengguna telah merasa puas atau tes berjalan dengan lancar / sukses.

Alat bantu: Komparator.

Data: Formalitas dokumen.

Definisi *Test Case*

Test case merupakan suatu tes yang dilakukan berdasarkan pada suatu inisialisasi, masukan, kondisi ataupun hasil yang telah ditentukan sebelumnya.

Guna test case

Untuk melakukan testing kesesuaian suatu komponen terhadap spesifikasi – *Black Box Testing*.

Untuk melakukan testing kesesuaian suatu komponen terhadap disain – *White Box Testing*.

White Box Testing

Kadang disebut juga *glass box testing* atau *clear box testing*, adalah suatu metode disain *test case* yang menggunakan struktur kendali dari disain prosedural.

Pendekatan secara internal yaitu memeriksa spesifikasi atau komponen produk.

Mengapa white box testing

Banyak error dalam “special case” code yang jarang dieksekusi

Control Flow tidak dapat diprediksi secara akurat dalam black-box testing.

Type error dapat terjadi dimana saja !

Kelebihan white box testing

Semua jalur (*path*) yang independen/terpisah dapat dites setidaknya sekali tes.

Semua logika keputusan dapat dites dengan jalur yang salah dan atau jalur yang benar.

Semua *loop* dapat dites terhadap batasannya dan ikatan operasionalnya.

Semua struktur internal data dapat dites untuk memastikan validitasnya.

Flow Graph (alur Logika)

Nodes (titik), mewakili pernyataan (atau sub program) yang akan ditinjau saat eksekusi program.

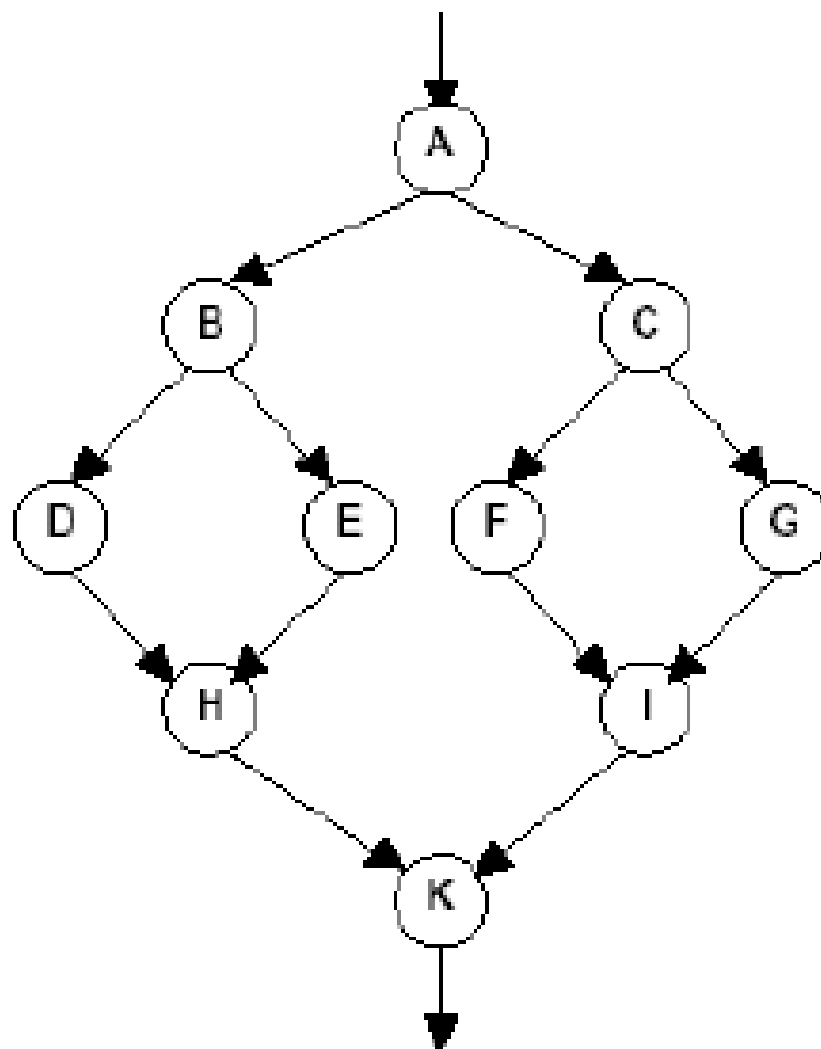
Edges (anak panah), mewakili jalur alur logika program untuk menghubungkan satu pernyataan (atau sub program) dengan yang lainnya.

Branch nodes (titik cabang), titik-titik yang mempunyai lebih dari satu anak panah keluaran.

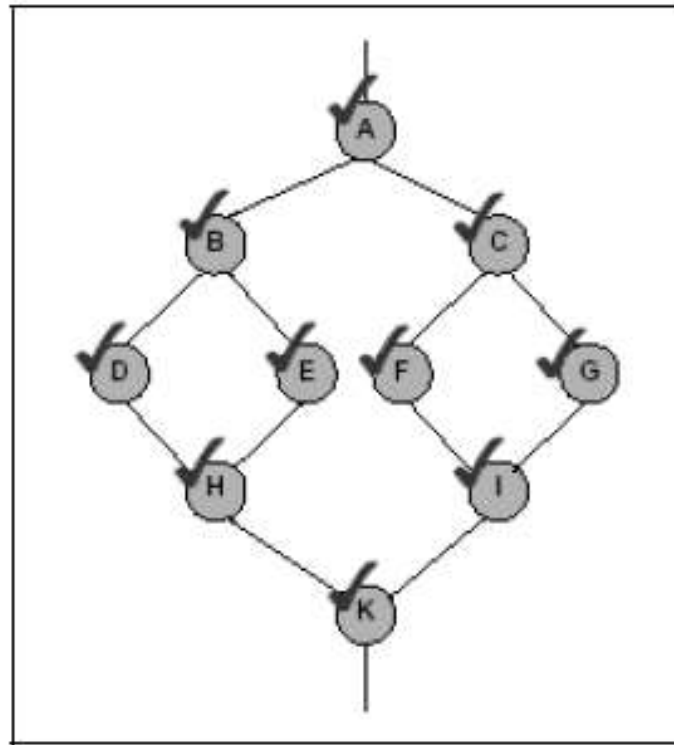
Branch edges (anak panah cabang), anak panah yang keluar dari suatu cabang

Paths (jalur), jalur yang mungkin untuk bergerak dari satu titik ke lainnya sejalan dengan keberadaan arah anak panah.

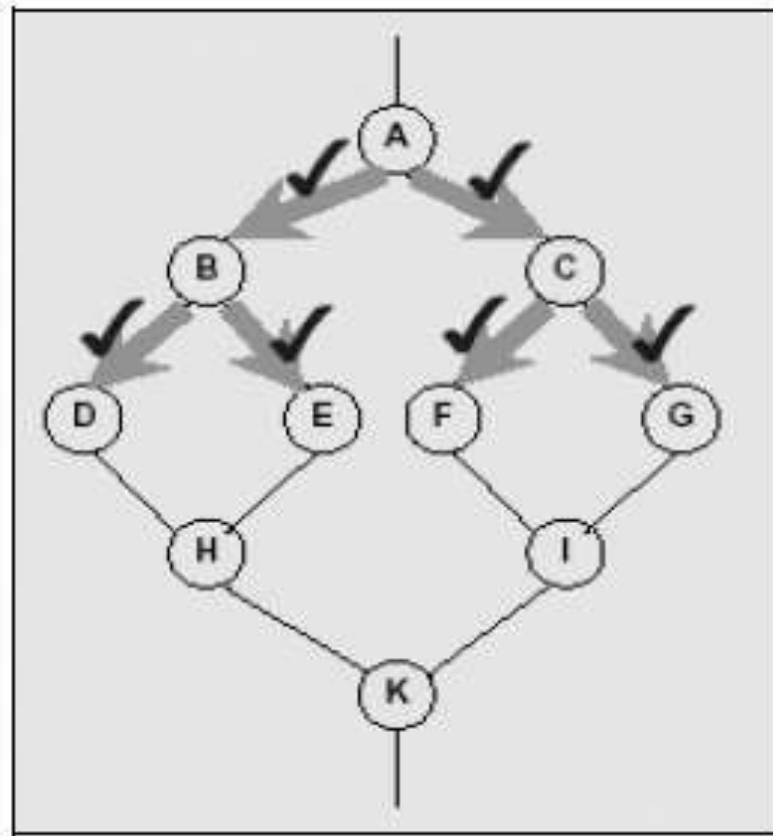
```
if A then
  if B then
    D
  else
    E
  end if;
  H
else
  if C then
    F
  else
    G
  end if;
  I
end if
```



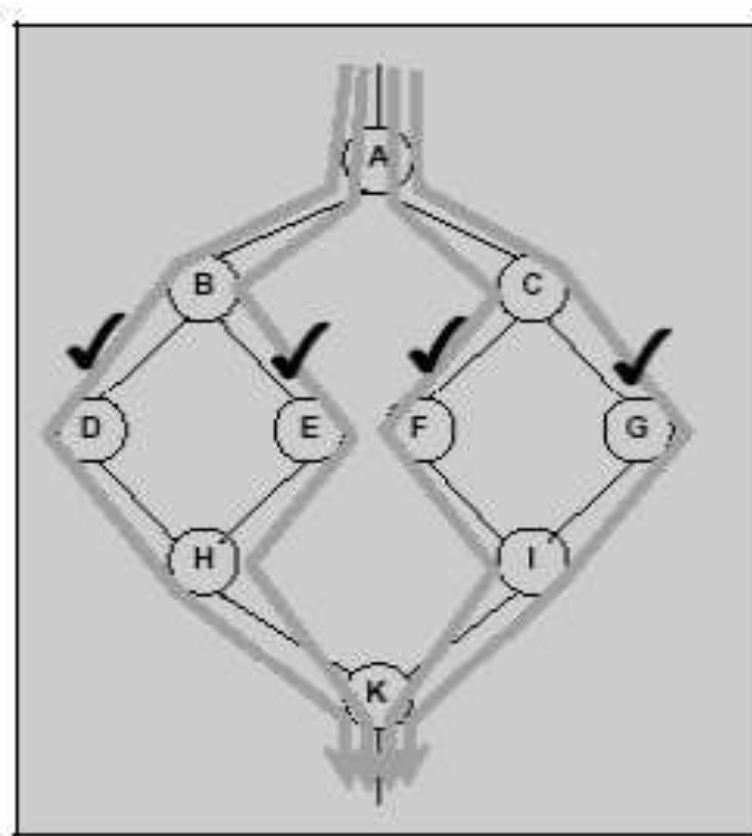
Cakupan pernyataan



Cakupan cabang

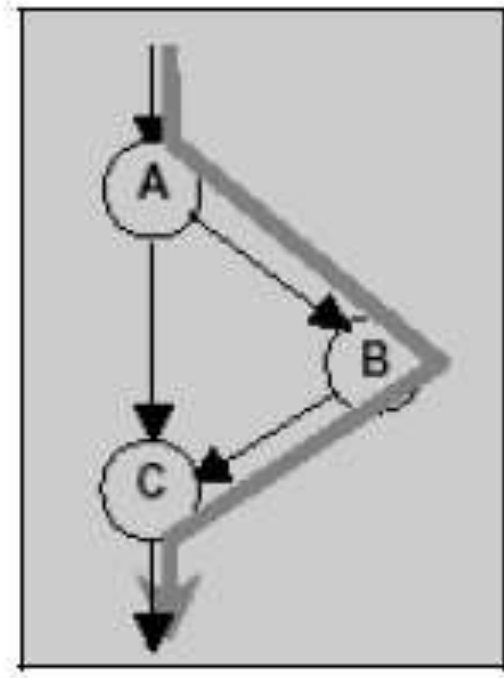
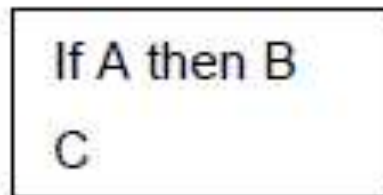


Cakupan jalur



Perbedaan cakupan pernyataan, cabang dan jalur

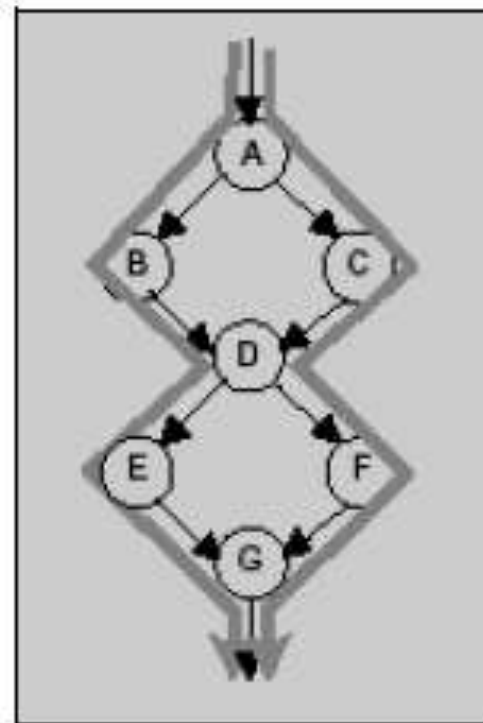
Contoh cakupan cabang 100 % namun cakupan jalur tidak 100 %.



Perbedaan cakupan pernyataan, cabang dan jalur

Contoh anak panah cabang 100 % namun cakupan jalur tidak 100 %.

If A then B else C
If D then E else F



Basis Path Testing

Proposed by Tom McCabe

Disebut juga branch testing atau *control-flow testing*

Hadir dalam 2 bentuk :

Zero Path: Jalur penghubung yang tidak penting atau jalur pintas yang ada pada suatu sistem.

One Path: Jalur penghubung yang penting atau berupa proses pada suatu sistem.

Basis Path Testing

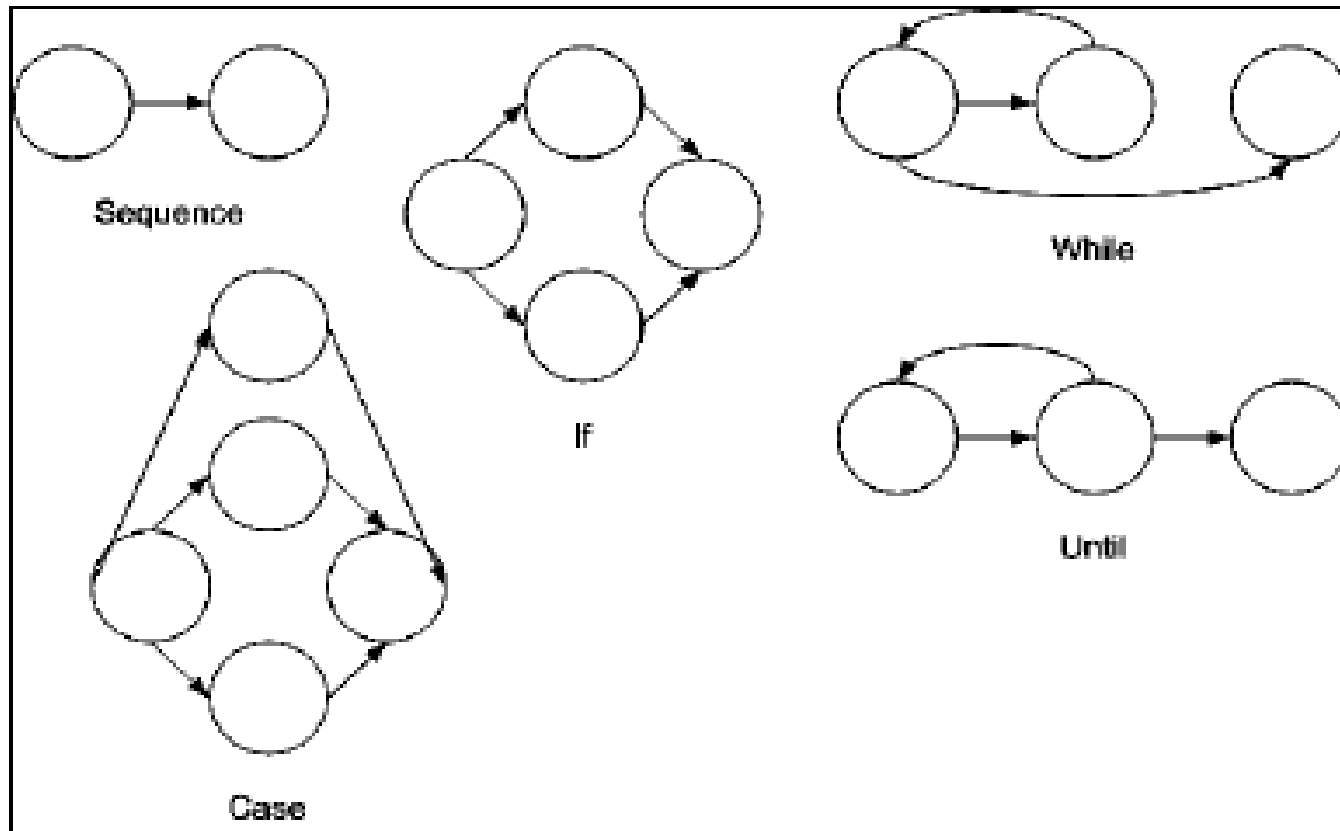
Bertujuan untuk melakukan pengukuran kompleksitas logikal dari rancangan prosedur dan menggunakannya sebagai guide untuk menentukan set dari path yang dieksekusi

Basic set akan dieksekusi oleh setiaps tatement paling tidak sekali

Isu

- Flow Graph Notation
- CyclomaticComplexity
- Deriving Test Cases
- Graph Matrices

Notasi flow graph



Notasi flow graph

Pada flow graph:

Panah disebut edges menggambarkan flow of control

Lingkaran disebut nodes, menggambarkan satu atau lebih aksi

Area yang dibatasi oleh edges dan nodes disebut regions

Predicate Nodes adalah nodes yang mengandung kondisi

Setiap procedural design dapat ditranslasikan ke flow graph notation

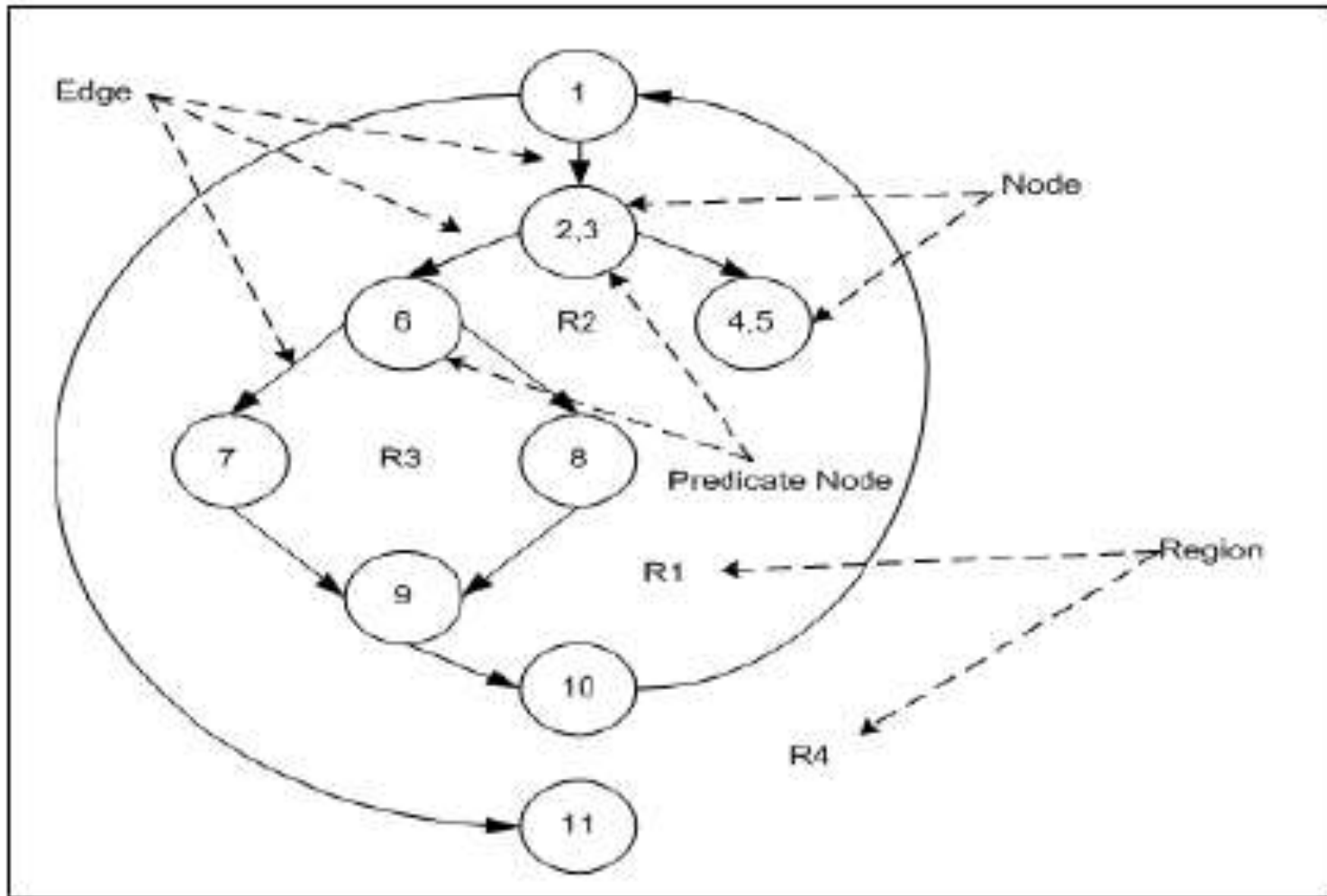
Cyclomatic Complexity

Memberikan ukuran kuantitatif dari kompleksitas logikal.

Nilainya memberikan jumlah dari independent path dalam basis set dan upper bound dari jumlah test untuk memastikan bahwa setiapstatement dieksekusi paling tidak satu kali.

Independent path adalah setiap path padaprogram yang mengenalkan paling tidak satu set baru statement yang sedang diproses atau kondisi baru (mis. Edge baru)

Flow graph untuk cyclomatic complexity



Jalur independen adalah tiap jalur pada program yang memperlihatkan 1 kelompok baru dari pernyataan proses atau kondisi baru.

$$\text{[Region / Complexity]} V(G) = E (\text{edges}) - N (\text{nodes}) + 2$$

$$V(G) = 11 - 9 + 2 = 4$$

$$V(G) = P (\text{predicate node}) + 1$$

$$V(G) = 3 + 1 = 4$$

Berdasarkan urutan alurnya, didapatkan suatu kelompok basis *flow graph* :

- Jalur 1 : 1-11
- Jalur 2 : 1-2-3-4-5-10-1-11
- Jalur 3 : 1-2-3-6-7-9-10-1-11
- Jalur 4 : 1-2-3-6-8-9-10-1-11

Tahapan test case dengan cyclomatic complexity

Gunakan disain atau kode sebagai dasar, gambarlah *flow graph*

Berdasarkan *flow graph*, tentukan *cyclomatic complexity*

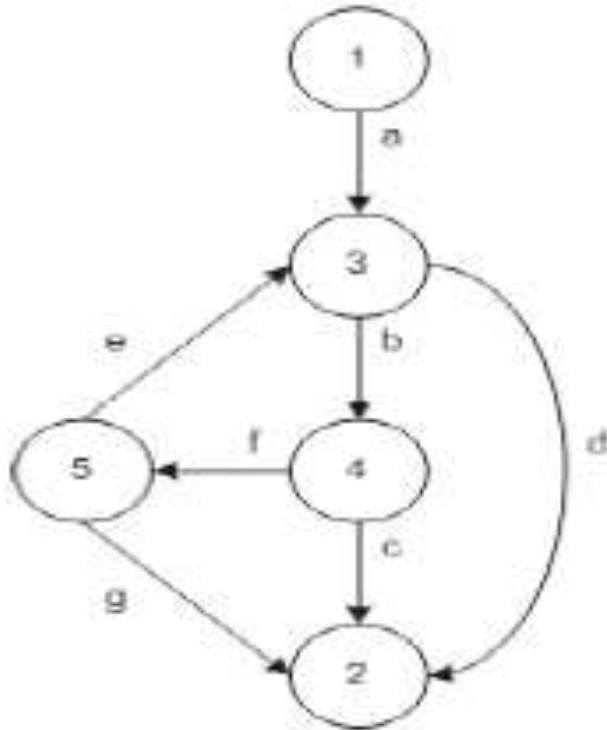
Tentukan kelompok basis dari jalur independen secara linier

Siapkan *test cases* yang akan melakukan eksekusi dari tiap jalur dalam kelompok basis

#Note: Beberapa path mungkin hanya dapat dieksekusi sbg bagian dari test yang lain.

Matriks graph

Flow Graph



Dihubungkan pada Node

Node

	1	2	3	4	5	Koneksi
1			1			$1 - 1 = 0$
2						
3		1		1		$2 - 1 = 1$
4		1			1	$2 - 1 = 1$
5		1	1			$2 - 1 = 1$

Cyclomatic Complexity = $3 + 1 = 4$

Connection Matrix

Matriks Graph

Dapat mengotomatisasi turunan dari flow graph dan menentukan set dari basis path

Graph matrix:

- Adalah bujursangkar dengan #sides sama dengan #nodes
- Baris dan kolom menggambarkan nodes
- Isi matriks menggambarkan edges
- Gunakan nilai 1 untuk menghitung cyclomatic complexity
 - Untuk setiap baris, jumlahkan nilai kolom dan kurangkan dengan 1
 - Jumlahkan totalnya dan tambahkan dgn 1 = CC

Matriks Graph

Link dapat diberikan bobot, sehingga dapat menentukan:

- Probabilitas bahwa sebuah link (edge) akan dieksekusi
- Waktu proses yang dihabiskan selama mengunjungi sebuah link
- Memory dan resource yang dibutuhkan selama mengunjungi sebuah link